

(find the file on the *LFY* website at http://linuxforu.com/article_source_code/aug11/clone.zip) allows the creation of threads, and is fundamentally similar to what the *pthread* library does. However, the direct use of *clone()* is discouraged, because if not used properly, it may crash the developed application. The syntax for calling *clone()* in a Linux program is as follows:

```
#include <sched.h>
int clone (int (*fn) (void *), void *child_stack, int flags,
void *arg);
```

The first argument is the thread function; it will be executed once a thread starts. When *clone()* successfully completes, “*fn*” will be executed simultaneously with the calling process. The next argument is a pointer to a stack memory for the child process. A step backward from *fork()*, *clone()* demands that the programmer allocates and sets the stack for the child process, because the parent and child share memory pages—and that includes the stack too. The child may choose to call a different function than the parent, hence needs a separate stack. In our program, we allocate this memory chunk in the heap, with the *malloc()* routine. Stack size has been set as 64KB. Since the stack on the x86 architecture grows downwards, we need to simulate it by using the allocated memory from the far end. Hence, we pass the following address to *clone()*:

```
(char*) stack + STACK
```

The next field, ‘*flags*’, is the most critical. It allows you to choose the resources you want to share with the newly created process. We have chosen *SIGCHLD* | *CLONE_FS* | *CLONE_FILES* | *CLONE_SIGHAND* | *CLONE_VM*, which is explained below:

- **SIGCHLD:** The thread sends a *SIGCHLD* signal to the parent process after completion. It allows the parent to *wait()* for all its threads to complete.
- **CLONE_FS:** Shares the parent’s file system information with its thread. This includes the root of the file system, the current working directory, and the *umask*.
- **CLONE_FILES:** The calling and caller process share the same file descriptor table. Any change in the table is reflected in the parent process and all its threads.
- **CLONE_SIGHAND:** Parent and threads share the same signal handler table. Again, if the parent or any thread modifies a signal action, it is reflected to both the parties.
- **CLONE_VM:** The parent and threads run in the same memory space. Any memory writes/mapping performed by any of them is visible to the other process. The last parameter is the argument to the thread function

(‘*threadFunction*’), and is a file descriptor in our case.

Please refer to the sample code implementation of LWP, in the accompanying *clone.c*.

The thread closes the file (*/dev/null*) opened by the parent. As the parent and this thread share the file descriptor table, the file close operation will reflect in the parent context also, and a subsequent file *write()* operation in the parent will fail. The parent waits till thread execution completes (till it receives a *SIGCHLD*). Then, it frees the memory and returns.

Compile and run the code as usual; and it should be similar to what is shown below:

```
$gcc clone.c
$.a.out
Creating child thread
child thread entering
child thread exiting
Parent: child closed our file descriptor
$
```

Linux provides support for an efficient, simple, and scalable infrastructure for threads. It encourages programmers to experiment and develop their libraries using *clone()* as the core component.

Please share your suggestions/feedback at vishalkanauja@gmail.com or cjgiridhar@gmail.com. 

References

1. [http://en.wikipedia.org/wiki/Clone_\(Linux_system_call\)](http://en.wikipedia.org/wiki/Clone_(Linux_system_call))
2. <http://linux.die.net/man/2/clone>
3. <http://www.linuxjournal.com/article/5211?page=0,0>
4. <http://www.evanjones.ca/software/threading.html>

Suggested reading

1. http://standards.ieee.org/findstds/interp/1003-1c-95_int/index.html
2. Sources of *pthread* implementation in “*glibc.so*”
3. <http://www.linux-mag.com/id/792/>

By: Vishal Kanauja and Chetan Giridhar

Vishal Kanauja is a Member, Technical Staff at NetApp, Bengaluru, India. He has written for LFY on virtual machines, Android OS, and Python. He is interested in compiler development, performance tuning, and algorithms. You can reach him at vishalkanauja@gmail.com.

Chetan Giridhar is a technology enthusiast and runs a website, TechnoBeans.com, which he updates with tools he has developed, apart from books, articles and publications written by him. He is an ardent traveller, and reads fiction. He also volunteers for NGOs that work towards the employment of specially-abled people. You can reach him at cjgiridhar@gmail.com.